Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 2



Manual de Usuario - St4ck Over Pig

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Octubre de 2026.

Índice

Descripción..... 3
1. Instalación y Requisitos del Sistema..... 3
2. Conocer la ventana principal..... 4
3. Abrir un proyecto..... 5
4. Abrir y editar archivos..... 5
5. Crear un archivo..... 6
6. Guardar el trabajo..... 7
7. Compilar y consultar resultados..... 8
8. Ejecutar el programa..... 10
    8.1 Entrada interactiva y detención..... 11
    8.2 Comprobación opcional con RARS externo..... 11
9. Paneles de herramientas..... 11
    9.1 Árbol AST..... 12
    9.2 Símbolos..... 12
    9.3 Pila..... 13
    9.4 Errores..... 14
10. Atajos de teclado..... 15
11. Situaciones frecuentes..... 15
12. Flujo de trabajo recomendado..... 16

Manual de Usuario: St4ck Over Pig

Descripción

Bienvenido al manual oficial del entorno de desarrollo **St4ck Over Pig**, una aplicación de escritorio diseñada para la creación, análisis y ejecución de programas bajo la sintaxis de tres lenguajes, siendo estos: PigLatin, Y, Zetariano. Permite organizar archivos mediante un árbol de proyecto, abrir varias pestañas de edición, guardar cambios, compilar el contenido y consultar representaciones en AST, C3D, C y Assembler por medio de RISC-V.

Este manual explica cómo operar la interfaz. La aplicación determina el lenguaje por la extensión del archivo; no hay un selector global de lenguaje. La interfaz admite las extensiones .pig, .y y .z. La compilación muestra errores y productos generados en distintas vistas.

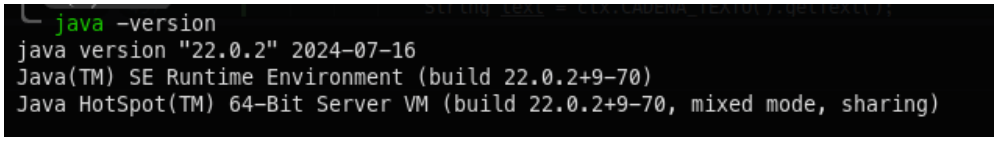
Los textos entre corchetes indican el lugar recomendado para insertar una captura propia. Sustituya cada marcador por una imagen de la aplicación en el estado descrito y, si lo desea, elimine el texto del marcador.

1. Instalación y Requisitos del Sistema

Al tratarse de una aplicación de escritorio, se debe de cumplir con los siguientes requisitos previos antes de iniciarla:

- **Requisito de Software:** Tener instalado **Java 22** (JDK 22) en tu sistema operativo. Puedes verificarlo abriendo tu terminal o consola y ejecutando el comando:

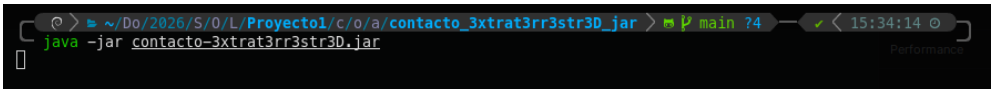
  java –version



  (Debe mostrar una versión compatible con Java 22).

- **Instalación y Ejecución:**
  1. Descarga el paquete de distribución de la aplicación de escritorio o el archivo ejecutable empaquetado (el archivo .jar ejecutable).
  2. Si ejecutas el archivo .jar directamente desde tu terminal, utiliza el siguiente comando:

  java -jar contacto_3xtr4t3rr3str3D.jar



La interfaz gráfica de la aplicación de escritorio se abrirá automáticamente en tu pantalla.

2. Conocer la ventana principal

• Barra superior: incluye los botones de nuevo, abrir y guardar; accesos al AST, símbolos, pila y errores; el selector Generar C (Cuartetas o Tripletes); el botón Herramientas y el botón Compilar.

• Árbol de proyecto, a la izquierda: presenta la carpeta raíz y los archivos de código que reconoce. Incluye filtro de búsqueda, botón Actualizar y botón para abrir una carpeta como proyecto. Un doble clic en un archivo lo abre en el editor.

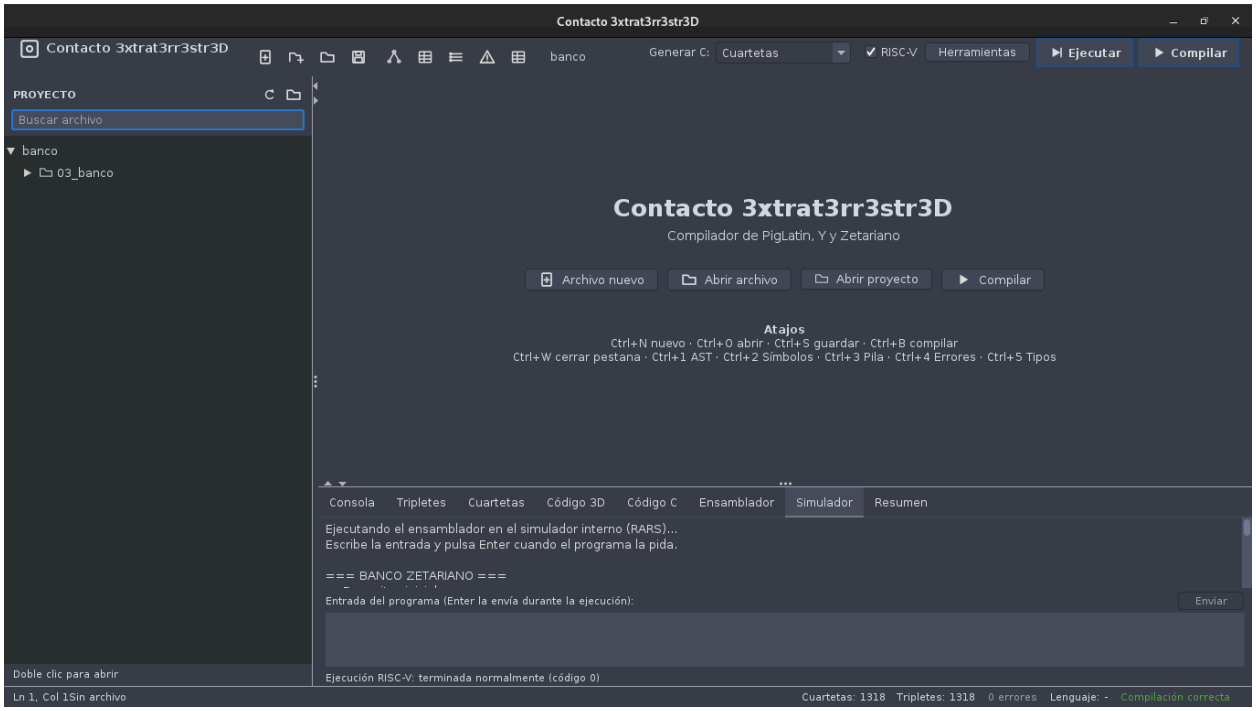
• Editor central: ofrece una pestaña por archivo, números de línea, resaltado visual y posición de cursor. El lenguaje se infiere de la extensión.

• Consolas inferiores: muestran Consola, Tripletes, Cuartetas, Código 3D, Código C y Resumen. Son áreas de consulta de solo lectura.

• Ventana Herramientas: es flotante y contiene las pestañas Árbol AST, Símbolos, Pila y Errores. Se abre desde Herramientas o desde los botones/atajos de vistas.

• Barra de estado: informa línea y columna, archivo activo, lenguaje, resultado de compilación, conteos de cuartetas/tripletes y cantidad de errores.

Los divisores entre árbol, editor y consolas se pueden arrastrar para ajustar el espacio. Algunos divisores incluyen control para contraer o expandir el panel.



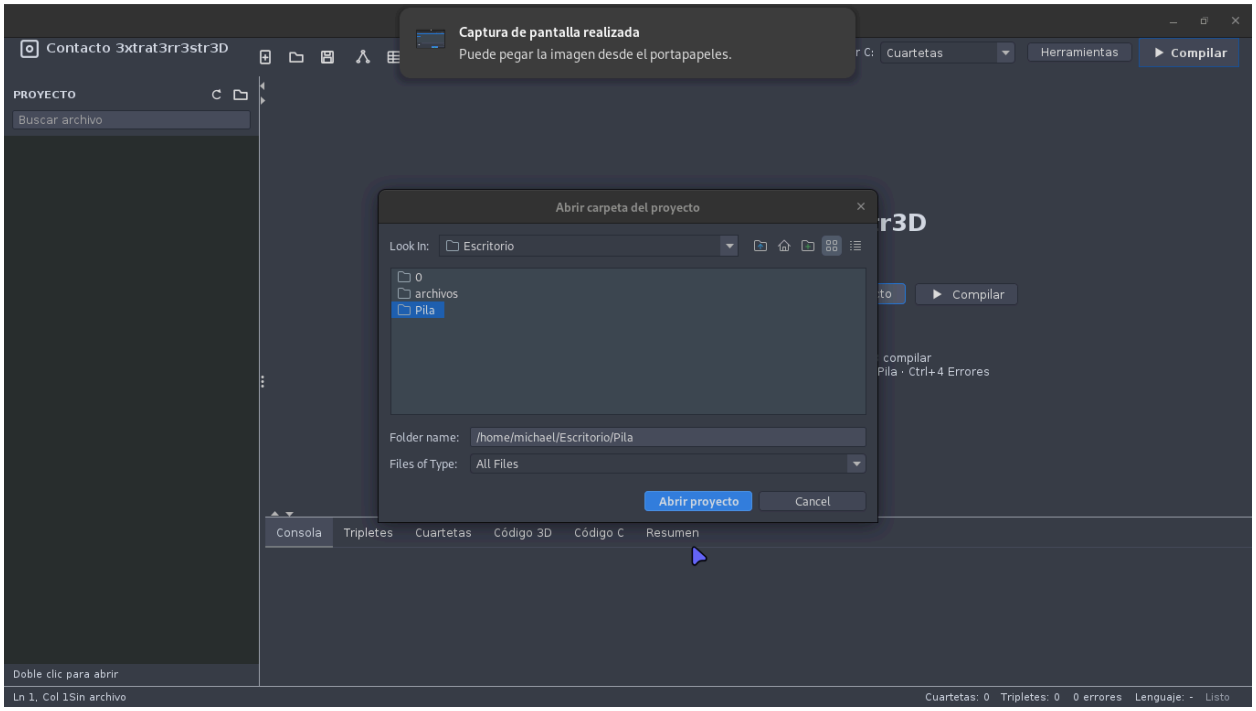
3. Abrir un proyecto

Un proyecto se abre seleccionando una carpeta que contiene los archivos del trabajo. Puede usar Abrir proyecto en la pantalla de bienvenida o el botón de carpeta de la cabecera del árbol. En el selector, navegue hasta la carpeta y confirme Abrir proyecto. El nombre de la carpeta aparece en la interfaz y su contenido se carga en el árbol.

En el árbol solo se muestran archivos con extensión reconocida (.pig, .y o .z); archivos de otros tipos se omiten. Si hay archivos ignorados, la aplicación puede informar cuáles no se incluyeron. Una carpeta que no contiene archivos fuente admitidos puede verse vacía.

Use el campo Buscar archivo para filtrar el árbol por texto. Borre el filtro para volver a mostrar todos los archivos. Use Actualizar después de cambios externos en el contenido de la carpeta para volver a leer el árbol.

Al hacer doble clic en un archivo fuente, se abre o activa su pestaña. Un único clic selecciona el elemento; la apertura se realiza con doble clic.



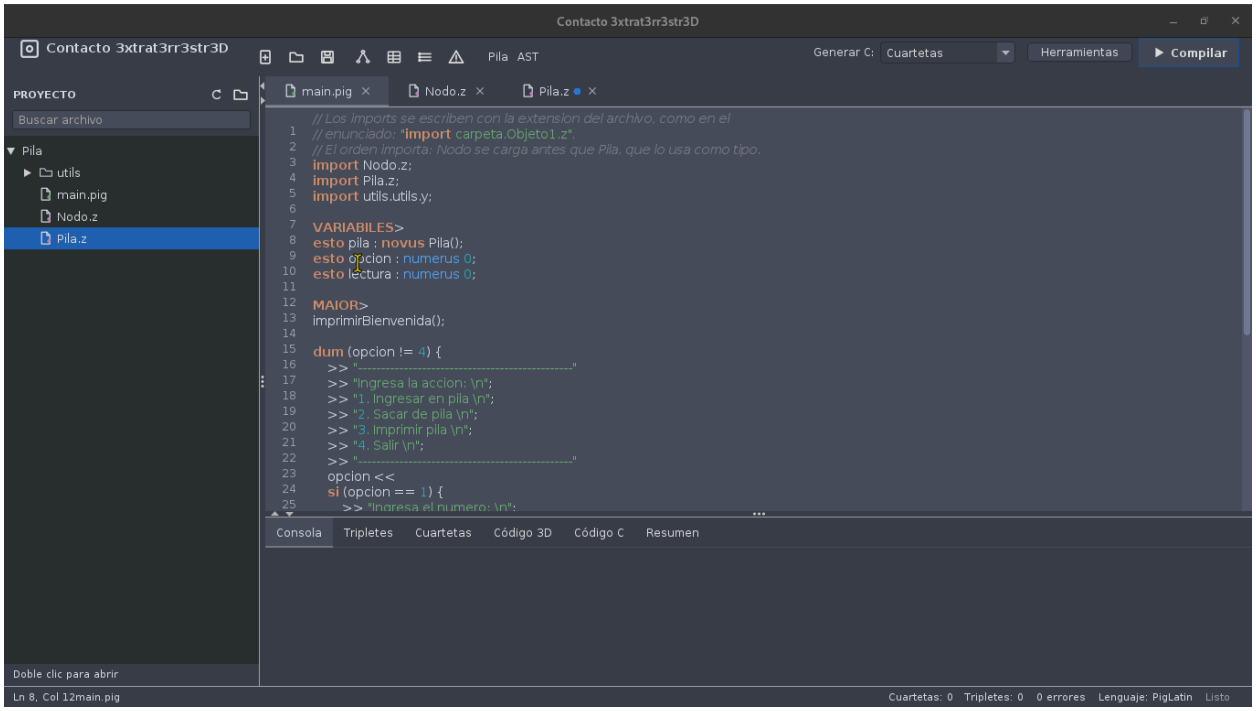
4. Abrir y editar archivos

Seleccione Abrir archivo (icono de carpeta/archivo o Ctrl+O), localice el archivo y ábralo. El selector filtra las extensiones de código aceptadas. Si el archivo seleccionado tiene una extensión no admitida, la interfaz avisa y no lo abre como fuente.

Si abre un archivo que está fuera del proyecto actual, la aplicación puede pedir cómo incorporarlo a la carpeta contenedora. Siga el diálogo mostrado para conservar el archivo dentro de la organización visible. La selección de un archivo en el árbol también lo abre en el editor.

Cada archivo abierto aparece en una pestaña. Cambie entre archivos seleccionando su pestaña; cierre la pestaña activa con Ctrl+W. Si cierra la última pestaña, vuelve a mostrarse la bienvenida. Al editar, se activa el resaltado asociado al lenguaje y la pestaña queda marcada como modificada. Revise el nombre de la pestaña/estado antes de cerrar para no perder cambios no guardados.

La barra inferior del editor muestra la línea y columna actuales, el nombre del archivo y el lenguaje detectado. La interfaz permite seleccionar y modificar texto como en un editor convencional.



5. Crear un archivo

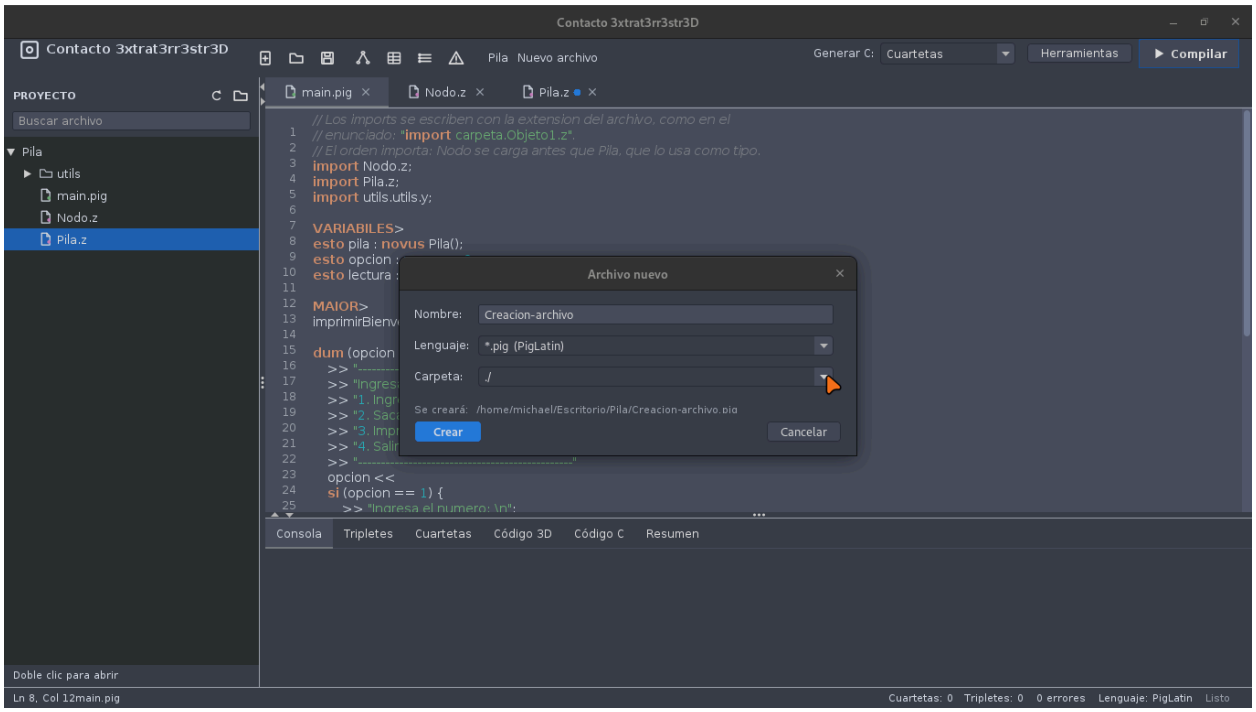
Pulse Archivo nuevo (Ctrl+N). Si no existe proyecto abierto, primero seleccione o cree una carpeta contenedora. En el diálogo Archivo nuevo complete los campos:

- Nombre: escriba el nombre base sin necesidad de añadir la extensión. Se propone un nombre libre del tipo archivoN.

- Lenguaje: seleccione la extensión correspondiente: .pig (PigLatin), .y (Y) o .z (Zetariano). La selección define el lenguaje y la extensión final.

- Carpeta: elija una de las carpetas listadas del proyecto. También puede seleccionar "… otra carpeta…" para buscar otra ubicación.

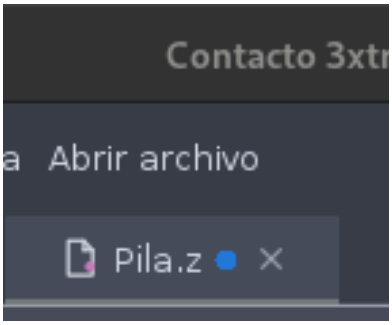
La vista previa muestra el nombre final que se creará. Pulse Crear para abrirlo en una pestaña nueva; pulse Cancelar o Escape para salir sin crear. El archivo creado se incorpora al árbol y queda listo para editar.



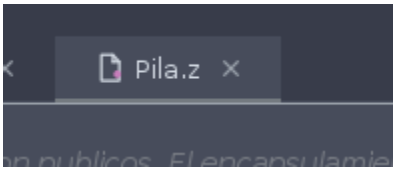
6. Guardar el trabajo

Use Guardar (Ctrl+S) para guardar la pestaña activa. Use Ctrl+Mayús+S para guardar todas las pestañas abiertas. Los cambios escritos en un editor permanecen como modificaciones locales hasta guardarlos; la compilación, sin embargo, puede utilizar el texto que está actualmente en el editor abierto, aunque todavía no se haya guardado.

Si un archivo nuevo fue creado desde la aplicación, guardar escribe el contenido en la ubicación elegida. Después de guardar, compruebe el estado o el mensaje de la consola para confirmar la acción.



Después de guardar, se guarda se quita el indicador de punto azul.



7. Compilar y consultar resultados

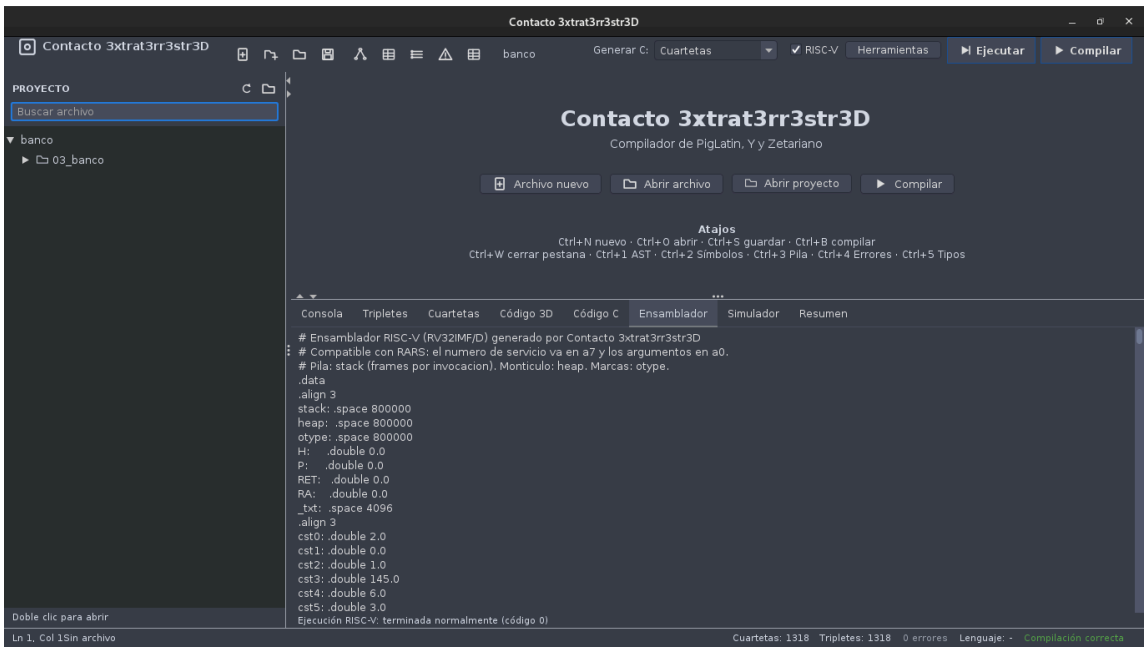
Pulse Compilar o use Ctrl+B. Cuando hay una carpeta de proyecto abierta, se compilan los archivos fuente reconocidos dentro del proyecto. Si existe main.pig, se toma como punto de partida y se procesa primero; los archivos importados se cargan durante ese proceso y luego se consideran los demás archivos del proyecto. Si no hay main.pig, se compilan los archivos según el orden indicado por la aplicación.

Sin carpeta de proyecto, se compilan los archivos guardados que estén abiertos en pestañas. Para un archivo abierto, la compilación toma el contenido que aparece en el editor, incluso si no se guardó aún. Los archivos no abiertos se leen desde disco.

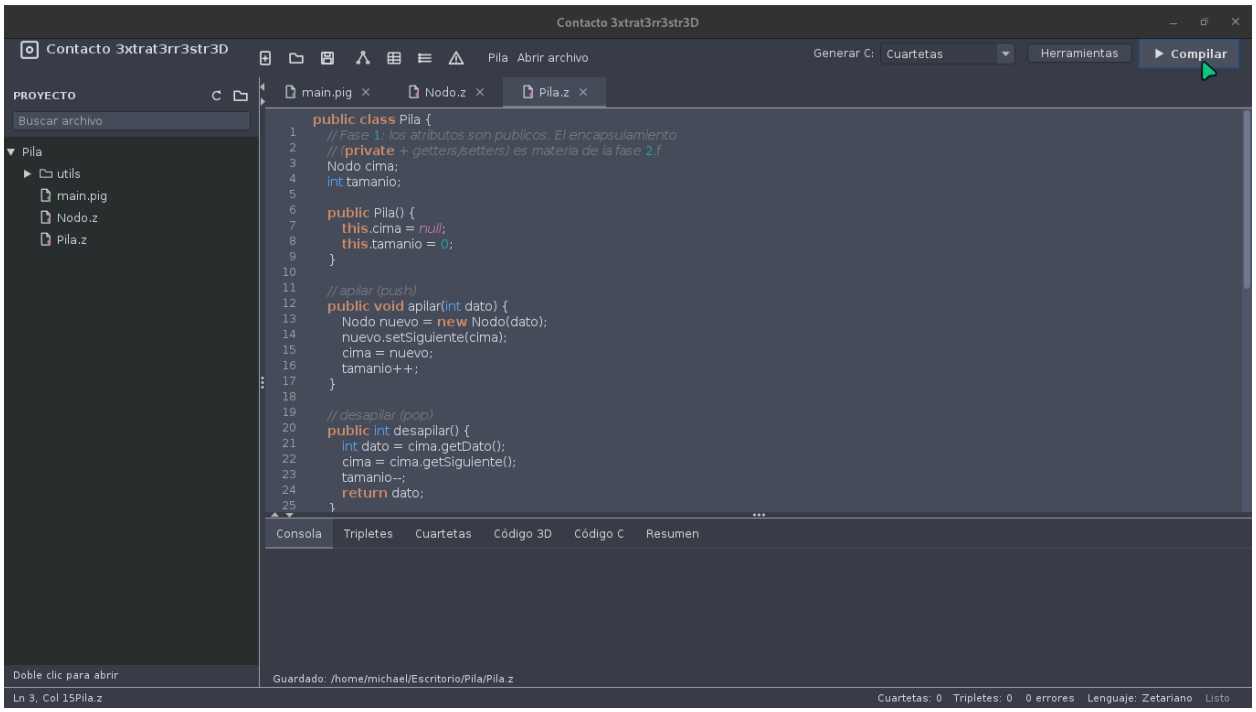
Antes de cada compilación se limpian las vistas de resultados anteriores. La pestaña Consola indica el alcance de la compilación y presenta el informe final. La barra de estado cambia a Compilación correcta o Compilación con errores e informa contadores. Si se encuentran errores, la ventana Herramientas se abre en Errores y, cuando el diagnóstico tiene línea y el archivo está abierto, el editor puede saltar a esa línea.

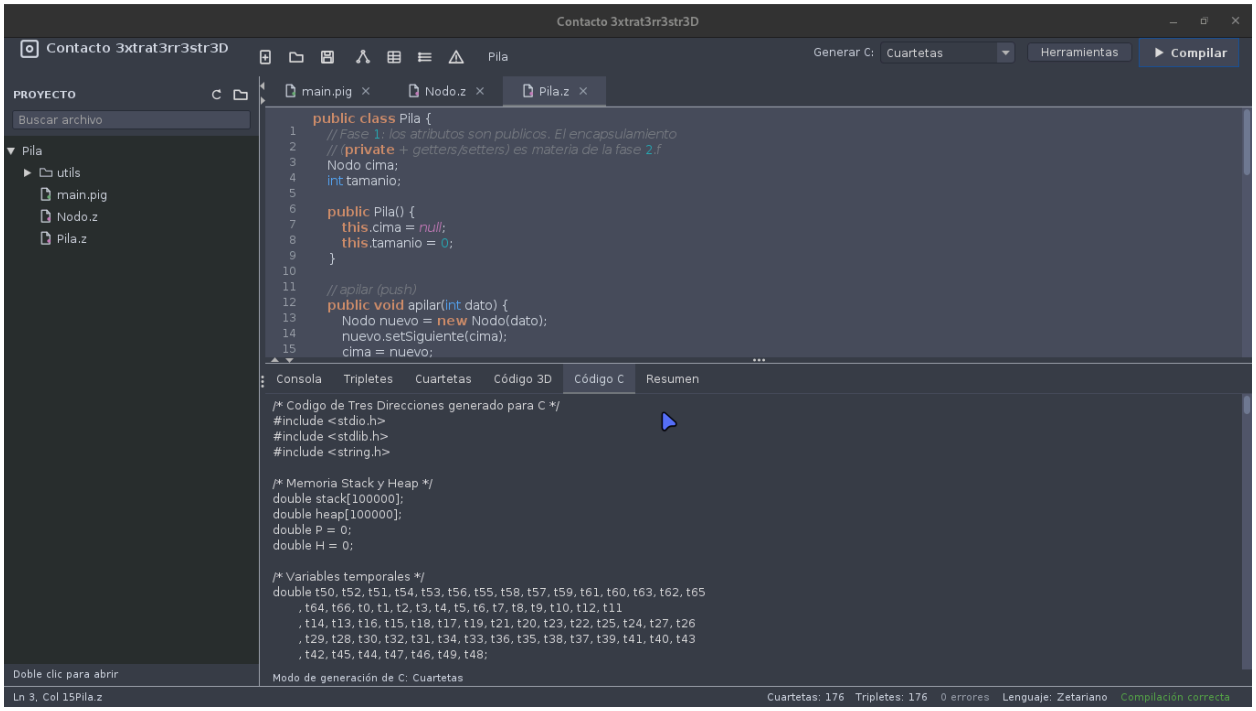
Para consultar las salidas, seleccione las pestañas inferiores:

- Consola: resumen del proceso y mensajes de compilación.
- Tripletes y Cuartetas: representaciones intermedias correspondientes al archivo de enfoque.
- Código 3D: código de tres direcciones.
- Código C: salida en C, en el modo de generación elegido.
- Ensamblador: ensamblador RISC-V del archivo de enfoque cuando la casilla RISC-V está activa.
- Resumen: texto traducido/resumen producido por la compilación.



El selector Generar C permite elegir Cuartetas o Tripletes como modo de generación de C. La opción se aplica a la compilación que se ejecute después del cambio. La casilla RISC-V agrega la traducción a ensamblador; si está desactivada, el backend RISC-V no se ejecuta. Las pestañas AST, Símbolos y Pila, al igual que las salidas de consola, muestran los resultados del archivo enfocado en el editor; la tabla Errores reúne los diagnósticos de la compilación del proyecto.





8. Ejecutar el programa

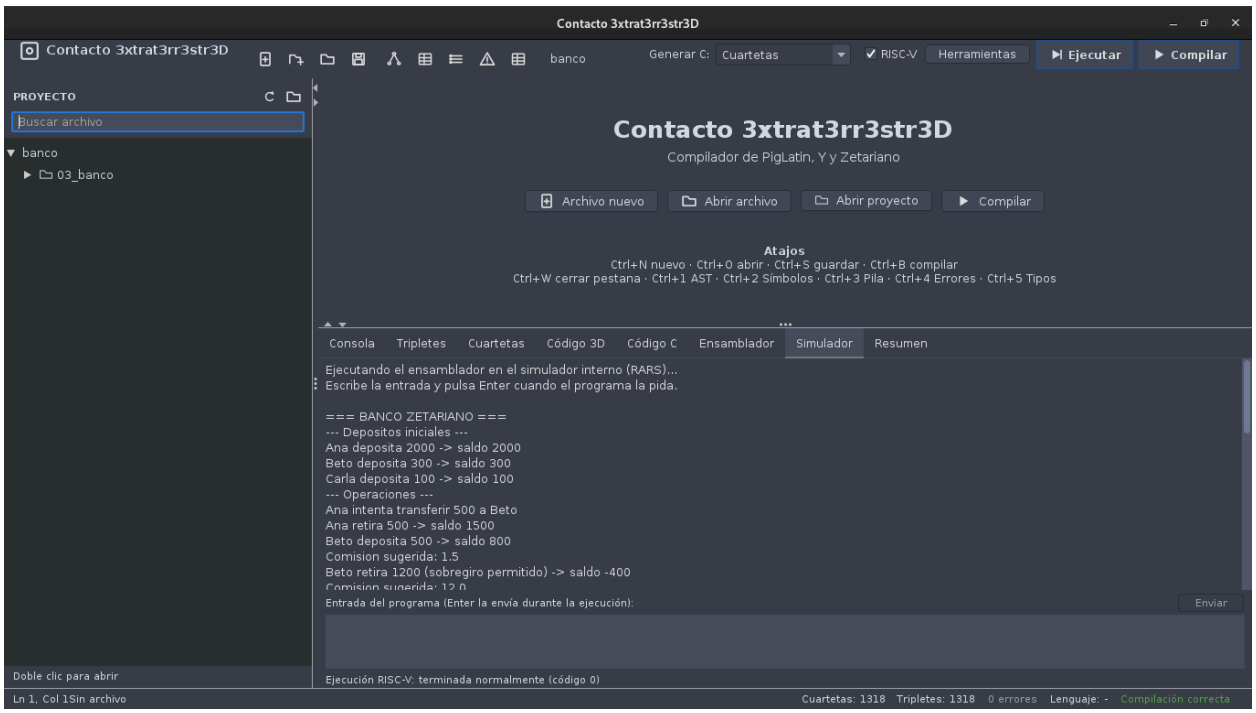
La ejecución se realiza dentro de la aplicación. El compilador genera el ensamblador RISC-V, lo carga en el simulador RARS 1.6 embebido y muestra la salida en la pestaña Simulador. No se abre una terminal ni se requiere un programa externo.

Para ejecutar: 1) marque la casilla RISC-V en la barra superior; 2) compile con Ctrl+B o con el botón Compilar; 3) pulse Ejecutar (o Ctrl+R); 4) revise la pestaña Simulador. Si la compilación tiene errores, la ejecución no comienza y la vista Errores muestra los diagnósticos.

Mientras el programa se ejecuta, el botón Ejecutar cambia a Detener. Pulse Detener para solicitar la parada; la aplicación cierra la entrada del simulador, marca la ejecución como detenida por el usuario y conserva la salida producida hasta ese momento.

La pestaña Simulador muestra la salida estándar, los mensajes de error, el estado de finalización, el tiempo transcurrido y la razón del cierre. Si el programa ejecuta una lectura, la ejecución queda bloqueada hasta que se envíe una línea desde el campo Entrada. Escriba el valor, envíelo con Enter o con el botón correspondiente y continúe. Un programa que no lee no requiere interacción.

El ensamblador generado se conserva en la pestaña Ensamblador. Puede copiarlo o guardarlo para revisarlo con RARS de escritorio si desea una comprobación adicional; la ejecución integrada sigue siendo la vía principal.



8.1 Entrada interactiva y detención

Cuando el programa llega a una lectura, el simulador espera una línea de entrada. El campo Entrada permanece disponible mientras la ejecución está activa. Si pulsa Detener durante esa espera, la entrada se cierra y el programa recibe el final de archivo (EOF), lo que permite desbloquear el simulador y finalizar la ejecución.

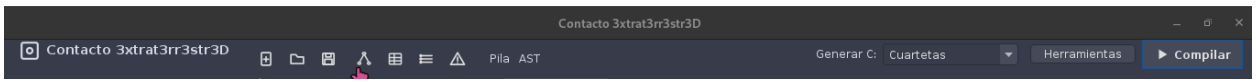
La salida se actualiza mientras el programa se ejecuta. Al terminar, la pestaña Simulador indica si la ejecución finalizó normalmente, con error o detenida por el usuario. Revise también la pestaña Errores cuando el cierre no sea normal.

8.2 Comprobación opcional con RARS externo

La pestaña Ensamblador contiene el texto .s que también se entrega al simulador interno. Si dispone de RARS de escritorio, puede copiar ese texto en un archivo .s y abrirlo con RARS para comparar resultados. Esta comprobación es opcional: no es necesaria para usar la ejecución integrada ni para calificar el proyecto.

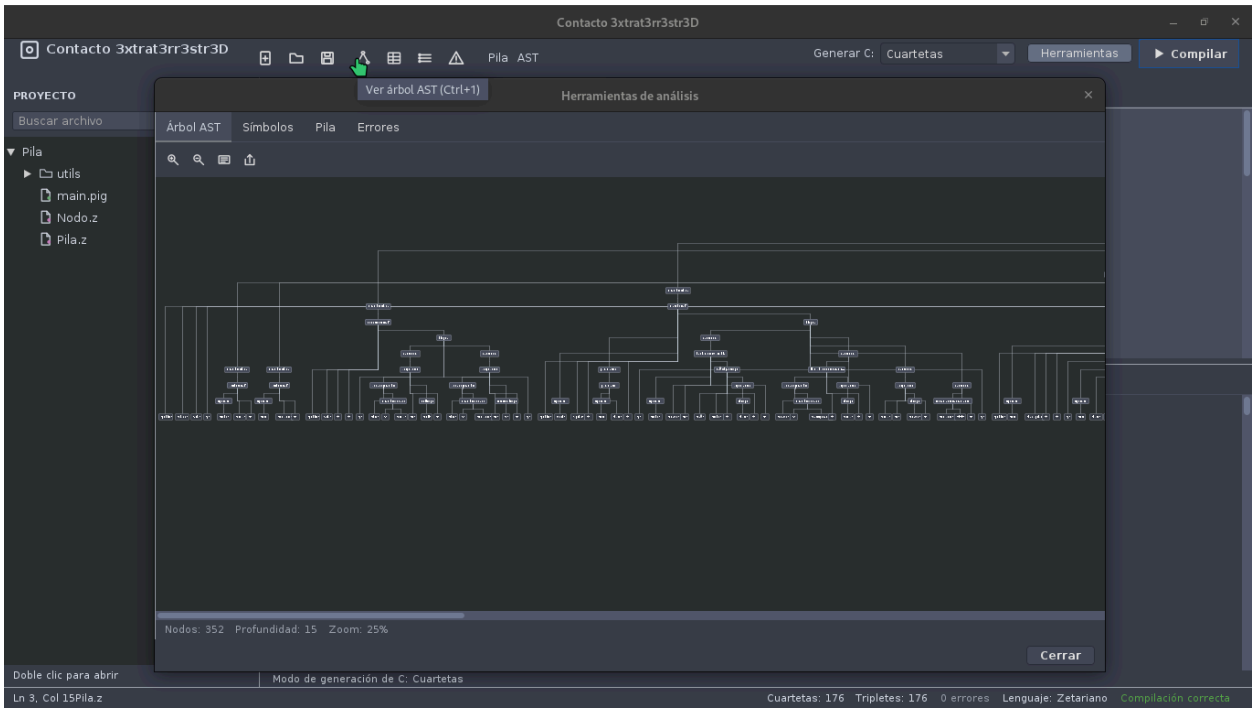
9. Paneles de herramientas

Abra la ventana flotante mediante Herramientas o el acceso directo específico. Las cuatro vistas están organizadas en pestañas. Cierre la ventana para volver al editor; puede reabrirla cuando necesite revisar resultados.



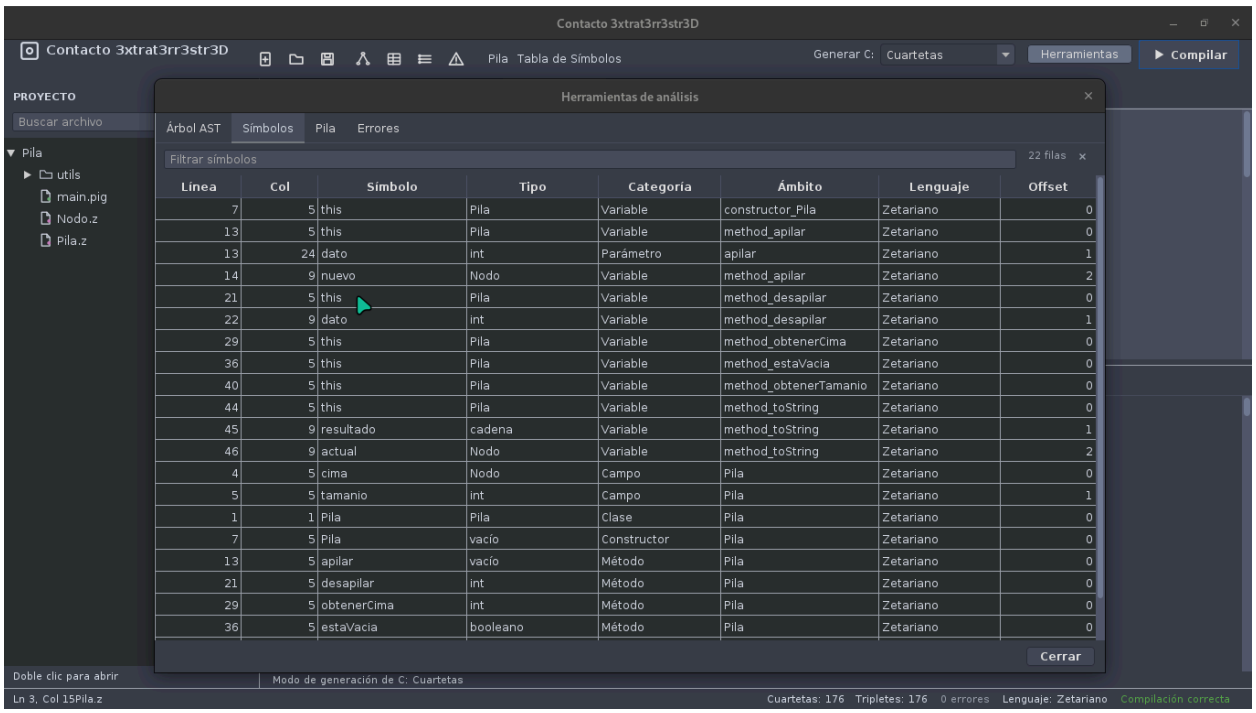
9.1 Árbol AST

Presenta el árbol de sintaxis abstracta del archivo de enfoque, si la compilación generó esa representación. Permite inspeccionar visualmente la estructura analizada. El contenido depende del archivo y de que el análisis haya podido avanzar.



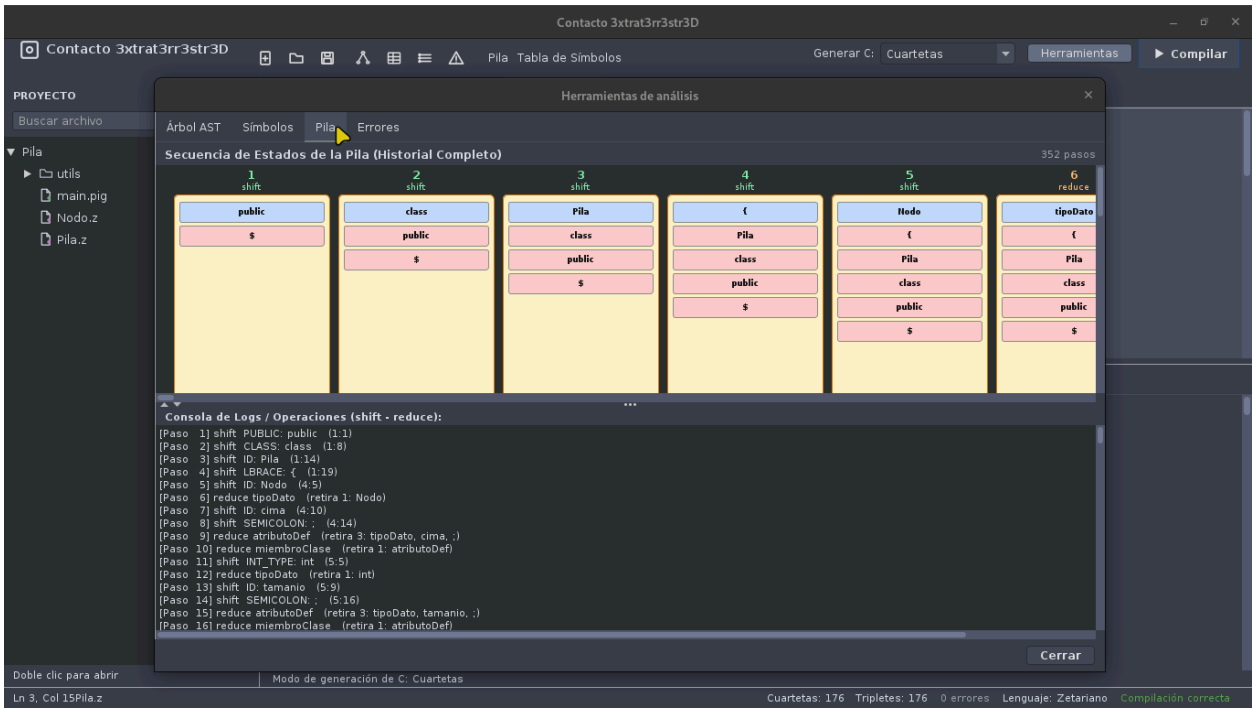
9.2 Símbolos

Muestra la tabla de símbolos del archivo de enfoque. Puede usar el filtro de la tabla para localizar elementos. Al hacer doble clic en un símbolo, la aplicación intenta llevar el editor a su declaración cuando existe una posición disponible.



9.3 Pila

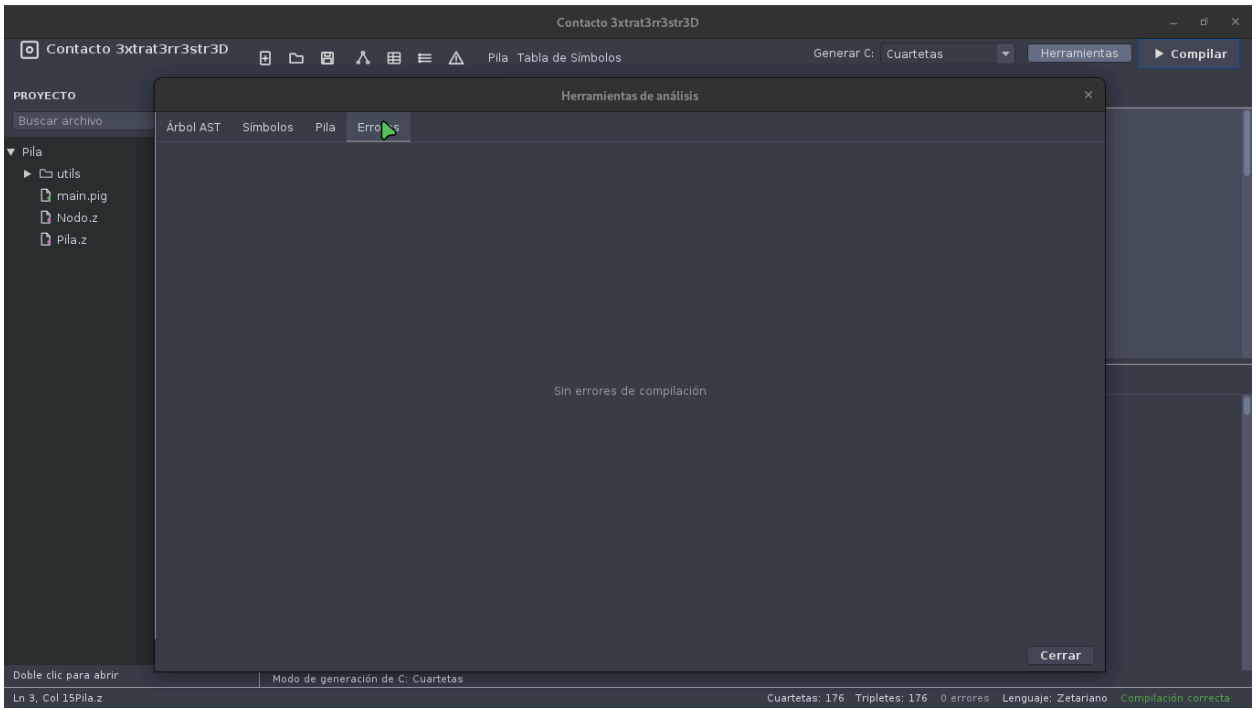
Muestra los pasos/estados del análisis asociados al archivo enfocado. Sirve para revisar la secuencia de procesamiento que la interfaz expone. La vista puede incluir una visualización y controles propios del panel.

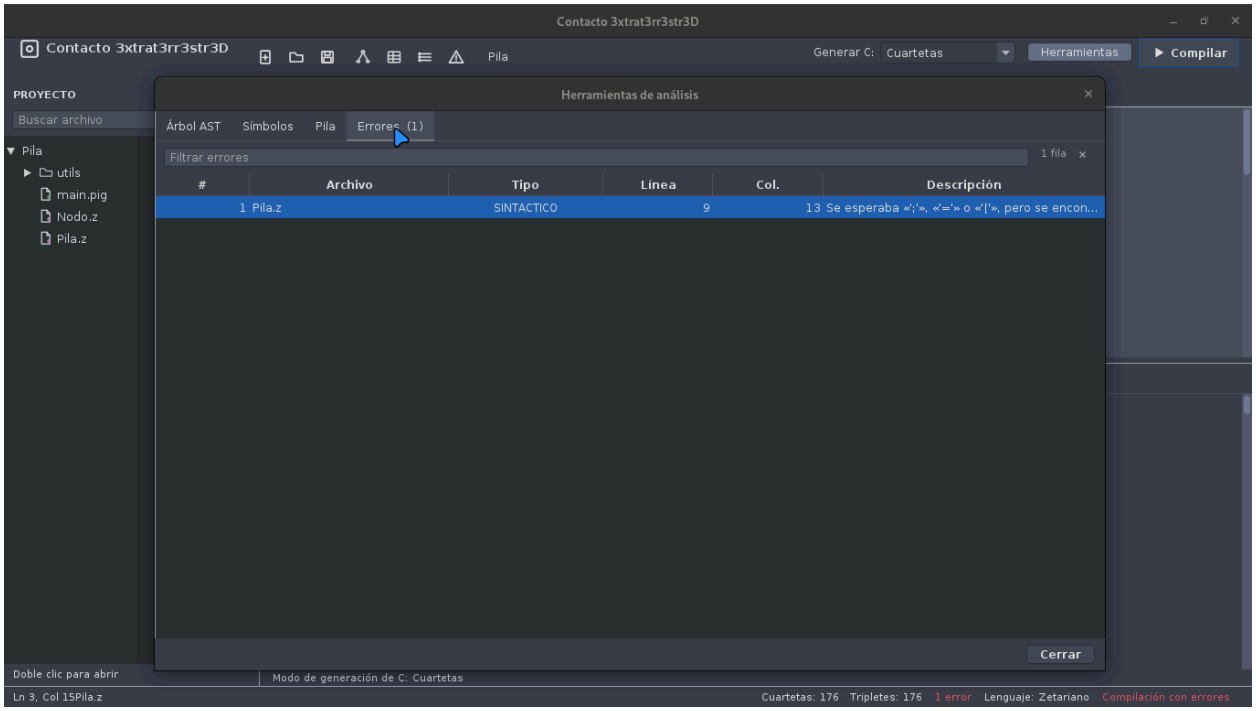


9.4 Errores

Presenta los errores recopilados durante la compilación del proyecto, con información de archivo y posición cuando está disponible. El contador de errores también se refleja en la pestaña y en la barra de estado. Haga doble clic en un error para abrir o enfocar su archivo y saltar a la línea reportada, si la ubicación es válida.

La tabla dispone de filtros para facilitar la búsqueda. Tras corregir el código, vuelva a compilar: la compilación nueva reemplaza los resultados anteriores.





10. Atajos de teclado

- Ctrl+N: crear archivo nuevo.
- Ctrl+O: abrir archivo.
- Ctrl+S: guardar pestaña activa.
- Ctrl+Mayús+S: guardar todas las pestañas.
- Ctrl+B: compilar.
- Ctrl+W: cerrar pestaña activa.
- Ctrl+1: mostrar Árbol AST.
- Ctrl+2: mostrar Símbolos.
- Ctrl+3: mostrar Pila.
- Ctrl+4: mostrar Errores.
- Escape: cerrar/cancelar el diálogo de archivo nuevo.

Los atajos Ctrl+1 a Ctrl+4 abren la ventana de herramientas y seleccionan la vista correspondiente. También puede acceder a esas vistas desde los botones de la barra superior.

11. Situaciones frecuentes

- El árbol está vacío: confirme que abrió la carpeta correcta, que contiene archivos .pig, .y o .z y que no hay un filtro activo. Pulse Actualizar para releerla.
- Un archivo no aparece: la interfaz ignora extensiones distintas de las admitidas. Compruebe la extensión final del nombre.
- No hay nada que compilar: abra una carpeta con archivos fuente o abra archivos guardados en pestañas. Sin proyecto, se compilan las pestañas abiertas que tienen archivo asociado.
- La compilación informa errores: abra Errores, lea el archivo/línea indicados, haga doble clic para navegar cuando sea posible, edite y compile de nuevo.
- No se ve el árbol o las consolas: arrastre los divisores para cambiar el espacio o utilice sus controles de expansión si están visibles.
- Una salida parece vacía: confirme que ejecutó Compilar y que está consultando la pestaña correcta. AST, símbolos, pila y salidas intermedias dependen del archivo enfocado.

• Los cambios no aparecen en disco: guarde la pestaña activa o todas las pestañas antes de revisar el archivo externamente.

12. Flujo de trabajo recomendado

• Abra la carpeta del proyecto y revise el árbol.

• Abra main.pig u otro archivo fuente con doble clic.

• Edite y guarde con Ctrl+S; use Ctrl+Mayús+S para guardar todo.

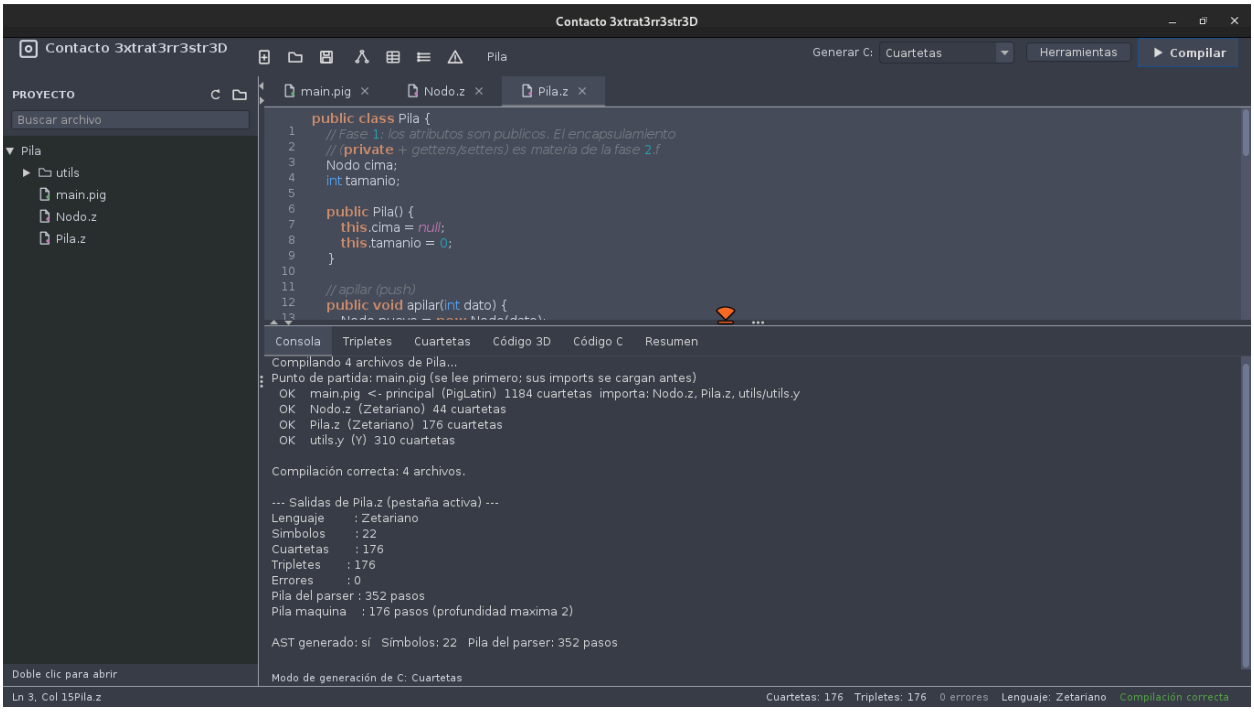
• Seleccione el modo de generación de C deseado.

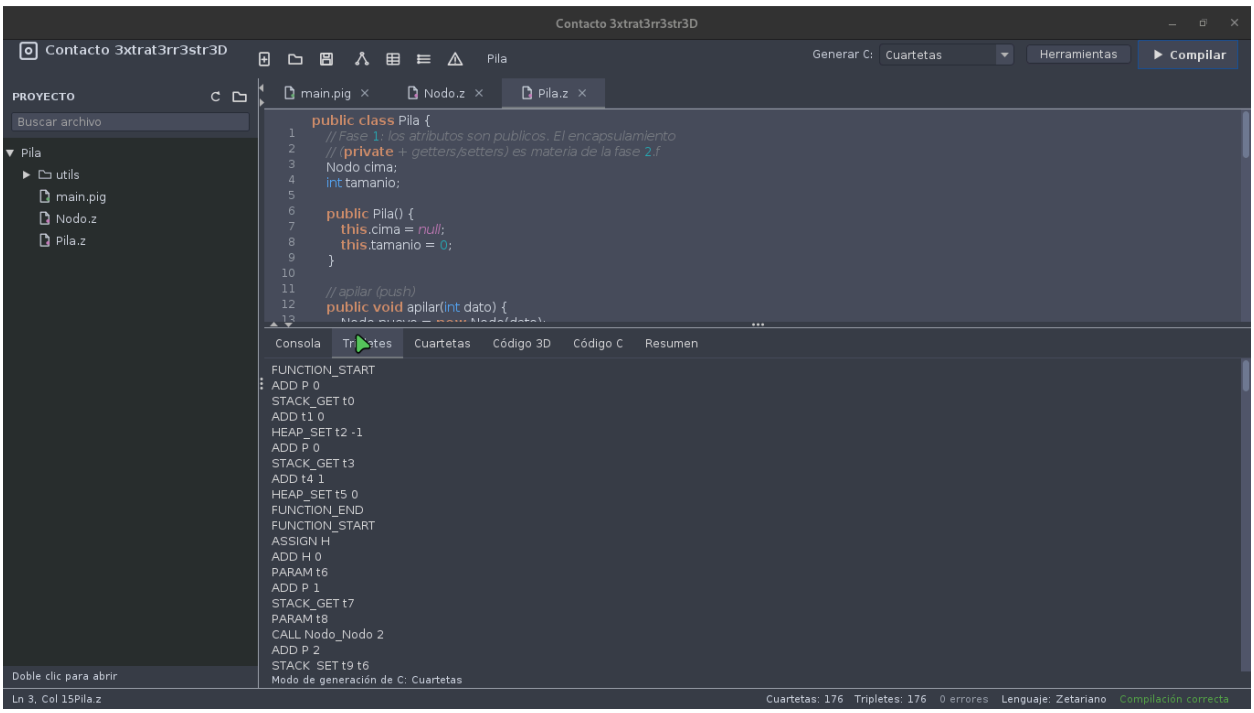
• Compile con Ctrl+B.

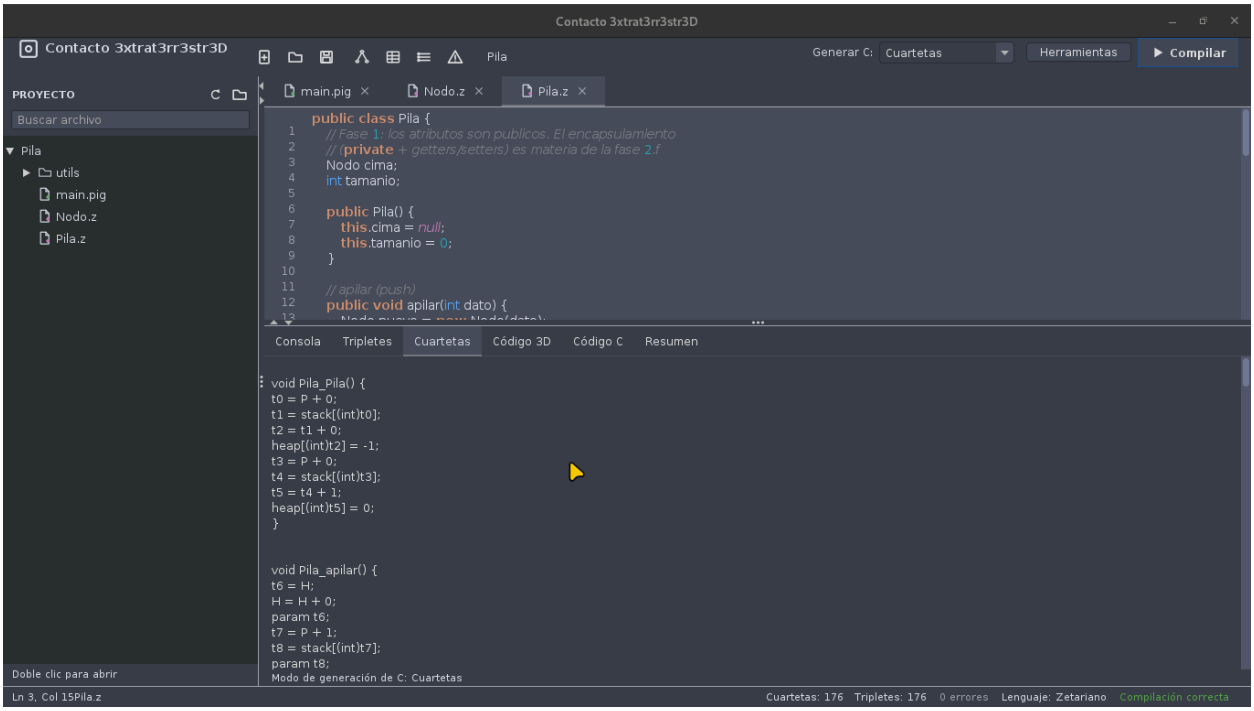
• Revise Consola y Resumen; consulte Tripletes, Cuartetas, Código 3D o Código C según lo que necesite.

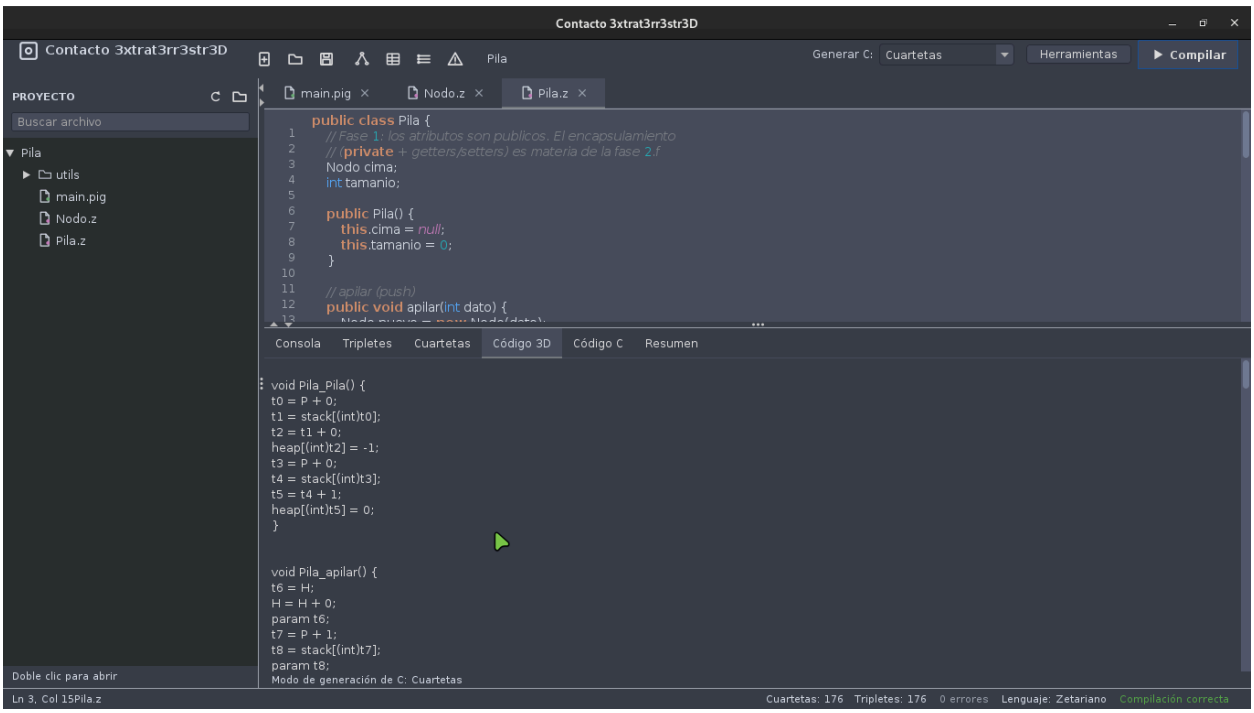
• Si hay errores, abra Errores, navegue a la ubicación, corrija y vuelva a compilar.

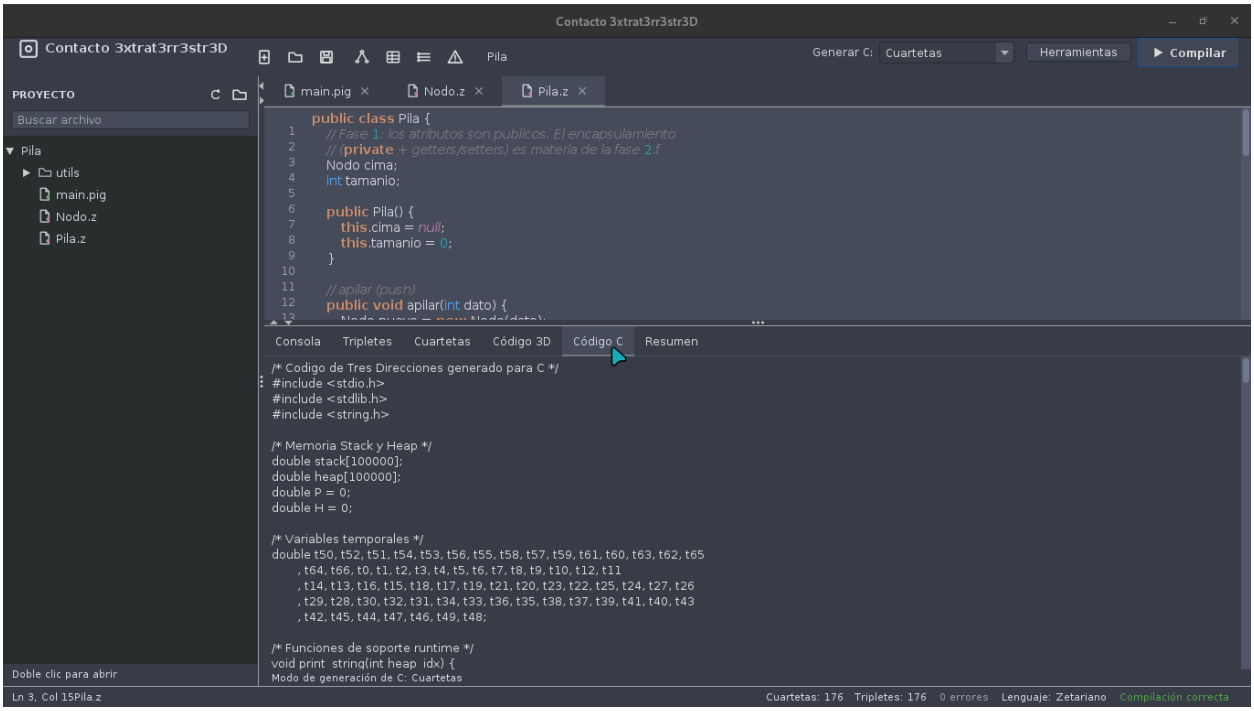
• Use AST, Símbolos y Pila cuando necesite inspeccionar el análisis del archivo enfocado.

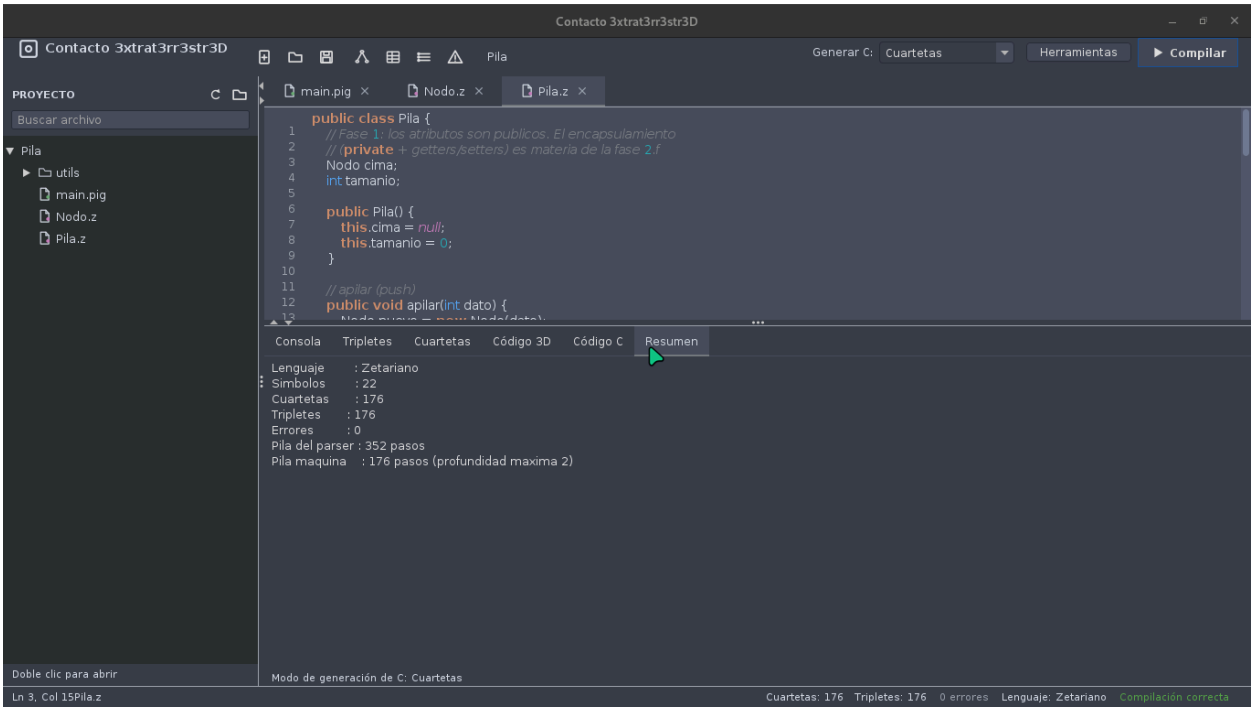



Contacto 3xtrat3rr3str3D
Pila
Generar C: Cuartetas
Herramientas
Compilar
PROYECTO
Buscar archivo
Pila
utils
main.pig
Nodo.z
Pila.z
main.pig
Nodo.z
Pila.z
public class Pila {
// Fase 1: los atributos son publicos. El encapsulamiento
// (private + getters/setters) es materia de la fase 2.f
Nodo cima;
int tamanio;
public Pila() {
this.cima = null;
this.tamanio = 0;
}
// apilar (push)
public void apilar(int dato) {
Consola
Tripletes
Cuartetas
Código 3D
Código C
Resumen
FUNCTION_START
ADD P 0
STACK_GET t0
ADD t1 0
HEAP_SET t2 -1
ADD P 0
STACK_GET t3
ADD t4 1
HEAP_SET t5 0
FUNCTION_END
FUNCTION_START
ASSIGN H
ADD H 0
PARAM t6
ADD P 1
STACK_GET t7
PARAM t8
CALL Nodo_Nodo 2
ADD P 2
STACK_SET t9 t6
Modo de generación de C: Cuartetas
Doble clic para abrir
Ln 3, Col 15Pila.z
Cuartetas: 176
Tripletes: 176
0 errores
Lenguaje: Zetariano
Compilación correcta


Contacto 3xtrat3rr3str3D
Pila
Generar C: Cuartetas
Herramientas
Compilar
PROYECTO
Buscar archivo
Pila
utils
main.pig
Nodo.z
Pila.z
main.pig
Nodo.z
Pila.z
public class Pila {
// Fase 1: los atributos son publicos. El encapsulamiento
// (private + getters/setters) es materia de la fase 2.f
Nodo cima;
int tamanio;
public Pila() {
this.cima = null;
this.tamanio = 0;
}
// apilar (push)
public void apilar(int dato) {
Consola
Tripletes
Cuartetas
Código 3D
Código C
Resumen
void Pila_Pila() {
t0 = P + 0;
t1 = stack[(int)t0];
t2 = t1 + 0;
heap[(int)t2] = -1;
t3 = P + 0;
t4 = stack[(int)t3];
t5 = t4 + 1;
heap[(int)t5] = 0;
}
void Pila_apilar() {
t6 = H;
H = H + 0;
param t6;
t7 = P + 1;
t8 = stack[(int)t7];
param t8;
Modo de generación de C: Cuartetas
Doble clic para abrir
Ln 3, Col 15Pila.z
Cuartetas: 176
Tripletes: 176
0 errores
Lenguaje: Zetariano
Compilación correcta


Contacto 3xtrat3rr3str3D
Contacto 3xtrat3rr3str3D
Pila
Generar C: Cuartetas
Herramientas
Compilar
PROYECTO
Buscar archivo
Pila
utils
main.pig
Nodo.z
Pila.z
main.pig
Nodo.z
Pila.z
public class Pila {
// Fase 1: los atributos son publicos. El encapsulamiento
// (private + getters/setters) es materia de la fase 2.f
Nodo cima;
int tamanio;
public Pila() {
this.cima = null;
this.tamanio = 0;
}
// apilar (push)
public void apilar(int dato) {
Consola
Tripletes
Cuartetas
Código 3D
Código C
Resumen
void Pila_Pila() {
t0 = P + 0;
t1 = stack[(int)t0];
t2 = t1 + 0;
heap[(int)t2] = -1;
t3 = P + 0;
t4 = stack[(int)t3];
t5 = t4 + 1;
heap[(int)t5] = 0;
}
void Pila_apilar() {
t6 = H;
H = H + 0;
param t6;
t7 = P + 1;
t8 = stack[(int)t7];
param t8;
Modo de generación de C: Cuartetas
Doble clic para abrir
Ln 3, Col 15Pila.z
Cuartetas: 176
Tripletes: 176
0 errores
Lenguaje: Zetariano
Compilación correcta


Contacto 3xtrat3rr3str3D
Contacto 3xtrat3rr3str3D
Pila
Generar C: Cuartetas
Herramientas
Compilar
PROYECTO
Buscar archivo
Pila
utils
main.pig
Nodo.z
Pila.z
main.pig
Nodo.z
Pila.z
public class Pila {
// Fase 1: los atributos son publicos. El encapsulamiento
// (private + getters/setters) es materia de la fase 2.f
Nodo cima;
int tamanio;
public Pila() {
this.cima = null;
this.tamanio = 0;
}
// apilar (push)
public void apilar(int dato) {
Consola
Tripletes
Cuartetas
Código 3D
Código C
Resumen
/* Codigo de Tres Direcciones generado para C */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* Memoria Stack y Heap */
double stack[100000];
double heap[100000];
double P = 0;
double H = 0;
/* Variables temporales */
double t50, t52, t51, t54, t53, t56, t55, t58, t57, t59, t61, t60, t63, t62, t65
, t64, t66, t0, t1, t2, t3, t4, t5, t6, t7, t8, t9, t10, t12, t11
, t14, t13, t16, t15, t18, t17, t19, t21, t20, t23, t22, t25, t24, t27, t26
, t29, t28, t30, t32, t31, t34, t33, t36, t35, t38, t37, t39, t41, t40, t43
, t42, t45, t44, t47, t46, t49, t48;
/* Funciones de soporte runtime */
void print_string(int heap_idx) {
Modo de generación de C: Cuartetas
Doble clic para abrir
Ln 3, Col 15Pila.z
Cuartetas: 176
Tripletes: 176
0 errores
Lenguaje: Zetariano
Compilación correcta


Contacto 3xtrat3rr3str3D
Contacto 3xtrat3rr3str3D
Pila
Generar C: Cuartetas
Herramientas
Compilar
PROYECTO
Buscar archivo
Pila
utils
main.pig
Nodo.z
Pila.z
main.pig
Nodo.z
Pila.z
public class Pila {
// Fase 1: los atributos son publicos. El encapsulamiento
// (private + getters/setters) es materia de la fase 2.f
Nodo cima;
int tamanio;
public Pila() {
this.cima = null;
this.tamanio = 0;
}
// apilar (push)
public void apilar(int dato) {
Consola
Tripletes
Cuartetas
Código 3D
Código C
Resumen
Lenguaje : Zetariano
Simbolos : 22
Cuartetas : 176
Tripletes : 176
Errores : 0
Pila del parser : 352 pasos
Pila maquina : 176 pasos (profundidad maxima 2)
Doble clic para abrir
Modo de generación de C: Cuartetas
Ln 3, Col 15Pila.z
Cuartetas: 176 Tripletes: 176 0 errores Lenguaje: Zetariano Compilación correcta